Homework Assignment: Implement a Football Video Analytics System

Applied Neural Network
Instructor: Maryam Abdolali
KNTU, Spring 2026

1 Assignment Goal

In this homework, you will implement a complete football video analytics system from scratch in Python. The system must process a football match video, detect and track players and the ball, estimate team membership, map image-space positions to a top-down pitch view, and generate annotated output videos.

2 Expected Final System

The final system should read a football match video from disk, detect players, referees, goalkeepers, and the ball, and track these objects consistently across frames. It should also estimate team membership, identify pitch reference points, transform player and ball positions into a top-down pitch coordinate system, render meaningful visualizations, and save the processed outputs to disk.

3 Recommended Project Structure

Your repository should be organized using clear module boundaries. The following structure is recommended, but small variations are acceptable if your design remains clean and well documented.

```
football_analytics/
|-- main.py
|-- config.py
|-- requirements.txt
|-- models/
|-- data/
|-- tracking/
| |-- __init__.py
| `-- tracker.py
|-- team_assignment/
| |-- __init__.py
| `-- team_assigner.py
|-- transformation/
| |-- __init__.py
| `-- transformer.py
|-- visualization/
| |-- __init__.py
| `-- renderer.py
|-- utils/
    |-- __init__.py
     `-- video_utils.py
```

4 Module Responsibilities

4.1 `main.py`: Pipeline Orchestration

The `main.py` file should coordinate the full pipeline. It should load the configuration, read the input video, call each processing stage in the correct order, and save the final outputs. Keep this file simple: heavy logic should live inside the dedicated modules rather than inside the orchestration script.

4.2 `config.py`: Configuration Management

The `config.py` file should store project-wide settings such as input and output paths, model paths, confidence thresholds, visualization colors, frame-sampling choices, and batch-size values. Avoid hard-coded paths and magic numbers inside the core logic files.

4.3 `tracking/tracker.py`: Detection and Tracking

This module should implement object detection and multi-object tracking. A recommended class name is `Tracker`. The class should load the detection model, run inference on frames or frame batches, convert detections into a tracking-friendly format, assign stable object IDs across frames, separate object categories such as players, referees, goalkeepers, and the ball, and optionally serialize or cache tracking results.

4.4 `team_assignment/team_assigner.py`: Team Identification

This module should estimate team membership based on visual appearance. A recommended class name is `TeamAssigner`. The implementation should extract player crops from bounding boxes, focus on jersey-dominant regions such as the upper body, compute representative colors, cluster players into two team groups, cache team labels by track ID, and store team-color metadata for later visualization.

4.5 `transformation/transformer.py`: Pitch Coordinate Transformation

This module should map camera-view coordinates to a normalized top-down pitch view. The implementation should detect pitch landmarks or keypoints, match image-space points to standard pitch-space coordinates, estimate a homography or similar geometric transformation, project player and ball positions into tactical pitch coordinates, and handle unreliable or missing keypoints.

4.6 `visualization/renderer.py`: Rendering Outputs

This module should handle visual output generation. It should draw annotations on the original frames, display track IDs and team labels, render player and ball positions on a synthetic pitch, optionally blend the tactical view with match footage, and save annotated videos or sample result frames.

4.7 `utils/video_utils.py`: Video and Geometry Helpers

This module should contain reusable helper functions for video reading, video writing, basic bounding-box geometry, and simple frame blending. Suggested function signatures are shown below.

```
def read_video(video_path):
    pass
```

```python
def save_video(frames, output_path, fps):
    pass

def blend_frames(frame_a, frame_b, alpha=0.5):
    pass
```

5 Recommended Libraries

You may choose alternative tools, but the following stack is strongly recommended.

Library	**Purpose**
`python`	Main programming language.
`numpy`	Array and coordinate operations.
`opencv-python`	Video reading, video writing, drawing, and image processing.
`ultralytics`	YOLO-based object detection and keypoint inference.
`supervision`	Detection conversion, ByteTrack, annotation utilities, and visualization helpers.
`scikit-learn`	KMeans clustering for team assignment.
`pickle`	Optional caching of intermediate detections or tracking results.
`matplotlib`	Optional debugging plots and visual inspection.
`pytest`	Unit testing.
`pathlib`	Safer path management.
`dataclasses`	Clean storage of configuration or detection result objects.

6 Recommended Data Structures

Use object-oriented design where it improves clarity. The main classes in your project will likely be `Tracker`, `TeamAssigner`, `Transformer`, `ViewTransformer`, and `Renderer`. Tracking results may be stored as dictionaries indexed by frame number and track ID, or as dataclasses if you prefer a more structured design.

```
tracks = {
    "players": [
        {track_id: {"bbox": [x1, y1, x2, y2]}}
    ],
    "referees": [
        {track_id: {"bbox": [x1, y1, x2, y2]}}
    ],
    "ball": [
        {track_id: {"bbox": [x1, y1, x2, y2]}}
    ]
}
```

The exact structure is flexible, but it must be easy for later modules to reuse detection, tracking, team, and transformation results.

7 Implementation Stages

Implement the project in stages rather than attempting to build the full system at once. Each stage should produce a small, testable result before you move to the next one. This makes debugging easier because you can verify video loading, detection, tracking, team assignment, transformation, and visualization independently.

7.1 Stage 1: Video Input and Output

Begin by implementing the video utilities before adding any machine learning components. Your first goal is simple: read an input football video, store the frames in memory or stream them in order, and write them back to a new video file without changing the content. This confirms that the project can handle frame size, frame order, frames per second, and output encoding correctly.

Use `opencv-python` for this stage. The main OpenCV tools you will likely need are `cv2.VideoCapture` for reading frames, `cv2.VideoWriter` for saving videos, and simple frame operations such as resizing or copying when necessary. Use `pathlib` to manage input and output paths safely. Your `utils/video_utils.py` file should contain reusable functions such as `read_video`, `save_video`, and simple geometry helpers.

A good checkpoint for this stage is an output video that has the same duration, resolution, and approximate visual quality as the input video. If the saved video plays correctly, then the rest of the pipeline has a reliable input-output foundation.

7.2 Stage 2: Object Detection

After video input and output are working, add object detection. The objective of this stage is to locate relevant objects in each frame, especially players, referees, goalkeepers, and the ball. The detector should return bounding boxes, class labels, and confidence scores for each detected object.

Use `ultralytics` for YOLO-based inference. A typical implementation loads a model using `YOLO(model_path)` and then runs prediction on individual frames or batches of frames. If you use a custom-trained football model, place the model weights inside the `models/` directory and store the path in `config.py`. Use `numpy` to store bounding boxes and confidence values in a consistent format such as `[x1, y1, x2, y2]`. You may also use `supervision` to convert model predictions into a cleaner detection object that is easier to pass into tracking and visualization code.

At this stage, do not worry about whether the same player has the same identity across frames. The main goal is frame-level detection. A good checkpoint is a short annotated video or a few saved frames showing bounding boxes around players and the ball.

7.3 Stage 3: Multi-Object Tracking

Detection alone treats each frame independently, so the same player may appear as a new object in every frame. Tracking solves this by assigning stable IDs to objects over time. In this stage, your system should connect detections from consecutive frames so that each player or referee keeps the same track ID for as long as possible.

Use `supervision` for tracking, especially its `ByteTrack` integration. A common design is to run YOLO detections first, convert them into a `supervision.Detections` object, and then pass them to a tracker. The tracker output should be converted into your own project format so that later modules do not depend too heavily on one external library. Store tracking results in dictionaries, lists, or dataclasses indexed by frame number and object type.

This stage should also separate players, referees, goalkeepers, and the ball into different groups. Players and referees usually benefit from stable IDs, while the ball may require special handling because it is small, fast, and often missed. A good checkpoint is an annotated video where each player has a visible ID that remains reasonably stable across frames.

7.4 Stage 4: Team Assignment

Once players have stable track IDs, implement team assignment. The objective is to estimate which team each player belongs to based on visual appearance. In most football videos, jersey

color is the most useful signal, so your implementation should crop each detected player and focus on the upper-body region rather than the whole bounding box.

Use `opencv-python` and `numpy` to crop player regions and manipulate pixel values. Use `scikit-learn`, especially `KMeans`, to cluster representative player colors into two team groups. A simple approach is to remove grass-colored background pixels, focus on the top half of the bounding box, compute a dominant or average jersey color, and cluster these colors into two groups. Cache the assigned team for each track ID so that the team label remains stable even when a player is partially occluded or temporarily misdetected.

This stage does not need to be perfect, but it should be explainable. In your report, describe which part of the crop you used, how you extracted representative color, and how you handled noisy detections. A good checkpoint is an annotated video where players are colored or labeled according to their estimated team.

7.5 Stage 5: Pitch Keypoint Detection

To map objects from the broadcast camera view to a top-down pitch view, the system needs reference points on the field. In this stage, detect pitch landmarks such as corners, penalty-box points, center-line points, or other known field markings. These keypoints provide the correspondence between image coordinates and standard pitch coordinates.

Use `ultralytics` if you have a YOLO keypoint or pose-style model trained for pitch landmarks. You may also use `opencv-python` for classical image-processing support, such as line detection, thresholding, or visual debugging. Store detected keypoints with confidence scores so that unreliable points can be filtered before transformation. Use `numpy` arrays for the image-space points and their matching pitch-space coordinates.

A stable transformation usually requires at least four reliable point correspondences for homography estimation, but more points are better because they allow filtering and robustness checks. A good checkpoint is a debug frame where detected pitch keypoints are drawn on the original video frame with labels.

7.6 Stage 6: Coordinate Transformation

Coordinate transformation converts player and ball positions from camera-view coordinates into a normalized top-down pitch coordinate system. This is the stage that makes tactical visualization possible. Usually, the point projected for a player should be the bottom-center of the bounding box, because that point approximates the player's contact point with the ground.

Use `opencv-python` for geometric transformation. The most important functions are likely `cv2.findHomography` for estimating the mapping and `cv2.perspectiveTransform` for projecting points. Use `numpy` to prepare point arrays, validate shapes, and filter invalid coordinates. Your `Transformer` or `ViewTransformer` class should take image-space points and return pitch-space points in a consistent coordinate system, such as meters or normalized pixel coordinates on a synthetic pitch image.

Your code should handle missing or low-confidence keypoints gracefully. For example, if too few reliable keypoints are available in a frame, you can skip transformation for that frame, reuse the most recent valid homography, or mark the transformed coordinates as unavailable. A good checkpoint is a small set of player and ball points correctly appearing on a top-down pitch drawing.

7.7 Stage 7: Visualization

Visualization helps you verify whether the system works and makes the final output understandable. In this stage, render annotations on the original video and create a tactical pitch view. The original video should show useful information such as bounding boxes or ellipses,

track IDs, team colors, and ball location. The pitch view should show projected positions of players and the ball in a clean top-down format.

Use `opencv-python` for drawing shapes, text, and overlays. You may use `supervision` annotation utilities if they simplify bounding-box or label rendering. Use `matplotlib` only for optional debugging plots; the final video rendering should normally be done with OpenCV because it integrates naturally with frame-by-frame video writing. The `Renderer` class should keep drawing logic separate from detection, tracking, and transformation logic.

A good checkpoint is an output video that combines the match footage with clear annotations and, optionally, a small tactical pitch panel. The visualization should be useful for debugging, not only decorative.

7.8 Stage 8: Refinement, Testing, and Integration

The final stage turns a working prototype into a maintainable project. Review the code structure, remove duplicated logic, replace hard-coded values with configuration variables, and make sure each module has one clear responsibility. Integrate the full pipeline in `main.py`, but keep `main.py` short and readable.

Use `pytest` to test deterministic parts of the project, such as bounding-box center calculation, bounding-box width calculation, coordinate projection with known points, and helper functions that do not depend on large models. Use `pickle` only when caching is useful, for example to avoid rerunning expensive detections during development. Use `dataclasses` if they make detection results, track results, or configuration objects easier to understand.

A good final checkpoint is a complete run from input video to saved output videos, together with a short report explaining the architecture, library choices, limitations, and possible improvements.

7.9 Suggested Library Use by Stage

Stage	Recommended libraries and purpose
Video input/output	`opencv-python` for reading and writing videos; `pathlib` for file paths.
Object detection	`ultralytics` for YOLO inference; `numpy` for bounding-box arrays; `supervision` for detection conversion.
Tracking	`supervision` for `ByteTrack`; `numpy` for coordinate handling; optional `pickle` for caching tracks.
Team assignment	`opencv-python` for crops and color processing; `scikit-learn` for `KMeans`; `numpy` for pixel statistics.
Pitch keypoints	`ultralytics` for keypoint detection; `opencv-python` for visual checks and optional classical CV support.
Transformation	`opencv-python` for homography and perspective transform; `numpy` for point arrays.
Visualization	`opencv-python` and optional `supervision` for annotations; optional `matplotlib` for debugging.
Testing/refinement	`pytest` for tests; `dataclasses` for structured objects; `pickle` for development caches.

8 Suggested Development Order

A sensible development path is to start with video utilities, then add object detection, tracking, and annotations on the original frames. After that, implement team assignment, pitch keypoint

detection, coordinate transformation, pitch visualization, and finally the full integration in `main.py`. The last step should be cleanup of the project structure, configuration, and tests.